Prayog — Code Coverage Report

CS455 Deliverable 3 · Group 3 · 6 November 2026

1. Summary

Production coverage is now 85.4%, above the course's 80% minimum. It rose from 81.2% because the GPU subsystem's 389-test suite joined the repository, and because the backend was measured on Linux, as in CI, where nothing fails for platform reasons.

Component	Statements (production code)	Covered	Coverage	Suite	Where measured
Backend (`backend/`, without tests, migrations and experiments)	8,247	6,943	**84.2%**	878 tests: 867 pass, 11 skipped (Docker, run in CI), 0 fail	`python:3.12` Linux container, final build
Steward (`steward/src/prayog_steward`)	2,643	2,290	**86.6%**	222 Steward tests	Areen's run; the package did not change. The suite is not in this repository
CLI (`cli/src/prayog`)	1,281	1,166	**91.0%**	165 tests, all pass	`python:3.12` Linux container
All production Python	**12,171**	**10,399**	**85.4%**		

Raw reports: `evidence/coverage_backend_linux.xml`, `evidence/coverage_cli_linux.xml`, `evidence/cov_steward.xml`. Areen's earlier Windows measurement (backend 77.9%, 81.2% overall) remains in `evidence/cov_backend.xml` and `evidence/cov_backend_html/`. CI now enforces the 80% minimum on the backend with `--cov-fail-under=80`.

2. Backend coverage by package

Package	Statements	Covered	Coverage
`routers/gpu/`	256	256	100.0%
`schemas/gpu/`	277	277	100.0%
`logic/gpu/scheduler/`	1,809	1,792	99.1%
`schemas/`	648	626	96.6%
`node_agent/`	537	518	96.5%
`routers/`	368	335	91.0%
`logic/`	2,400	1,982	82.6%
`top-level modules`	1,274	806	63.3%
`integration/`	498	287	57.6%
`scripts/`	180	64	35.6%

3. Least-covered files (at least 30 statements)

File	Coverage	Why
`mlflow_storage_smoke.py`	0.0%	a CI smoke script, run as a separate CI step
`scripts/setup_steward.py`	0.0%	operational script, needs the Steward role
`mlflow_worker.py`	28.6%	the Docker execution path; runs in CI with `PRAYOG_RUN_DOCKER_SMOKE=1`
`scripts/seed_dev.py`	36.6%	operational script, used by every drill
`logic/mlflow_professor.py`	38.5%	PostgreSQL professor tests skip parts without a seeded supervision graph
`mlflow_outbox_worker.py`	46.0%	worker loop and retries
`integration/steward_bridge.py`	47.9%	most of it needs `STEWARD_ENABLED=true`, which the main suite turns off; covered by the Steward harness
`logic/diagnostic_links.py`	69.1%	error paths and rarely used branches
`logic/mlflow_handover.py`	71.4%	error paths and rarely used branches
`logic/access_request.py`	73.0%	error paths and rarely used branches

The scheduler gap is closed. Areen's measurement showed `logic/gpu/scheduler/reaper.py` at 0% and `logic/gpu/scheduler/` at 77.5%, because the GPU subsystem's tests had been held back for this deliverable. With them, `logic/gpu/scheduler/` is at 99.1%, and `reaper.py` at 100.0%.

4. Failing tests in the measured runs

All 13 backend failures and 3 CLI failures were inspected; every one is Windows-specific except one test-isolation issue in a new test, which was fixed and re-run.

Test(s)	Count	Cause	On Linux CI
`test_node_agent_units::SubprocessRunnerTests` (2 failures + 2 errors), `test_node_agent_runs` (3)	5 (+2 errors)	`os.killpg` does not exist on Windows; Windows locks files a running process holds (`WinError 32`)	pass
`test_node_agent_units::ResultShipperTests` (2), `test_mlflow_worker::test_output_artifact_must_be_regular_file_not_symlink`, CLI `test_folder` / `test_runs` symlink tests	5	creating symlinks needs a Windows privilege (`WinError 1314`)	pass
`test_artifact_storage` (2), CLI `test_credential_file_is_private`	3	POSIX permission bits (`0o600` read back as `0o666`)	pass
`test_mlflow_worker::test_extracts_archive_and_validates_relative_entrypoint`, `test_diagnostic_session_audit::test_consumption_requires_same_user_session_and_audits_success`	2	path separators (`WindowsPath` vs `PurePosixPath`, `\private\log.txt`)	pass
`test_concurrency::test_c3_parallel_passes_never_double_allocate_a_unit`	1	new D3 test: its last assertion did not expect the shared test database's other nodes to take its jobs	corrected (no product change); passes on the shared and the dedicated database

5. Suites not in the coverage totals

Suite	Where	Result	Note
Backend ↔ Steward integration (`backend/tests_steward_check`, 17 tests)	local harness, untracked	17 pass	the harness creates its own database and Steward role
Docker worker, node-agent container, API-to-worker e2e (11 tests)	CI only	green on PR #26	need Docker
Concurrency (6), load, failure drills, adversarial probes	`test_cases/gpu/test_concurrency.py`, `evidence/`	see reports 03–06	the drills and probes are scripts, not counted as coverage

6. Coverage gate

There is no coverage gate in CI today. Recommended: publish `coverage xml` from the Linux CI job (where nothing is skipped) and fail the job if production coverage drops below the current 78% for the backend, with the reaper tests (section 3) as the first step to raise it.